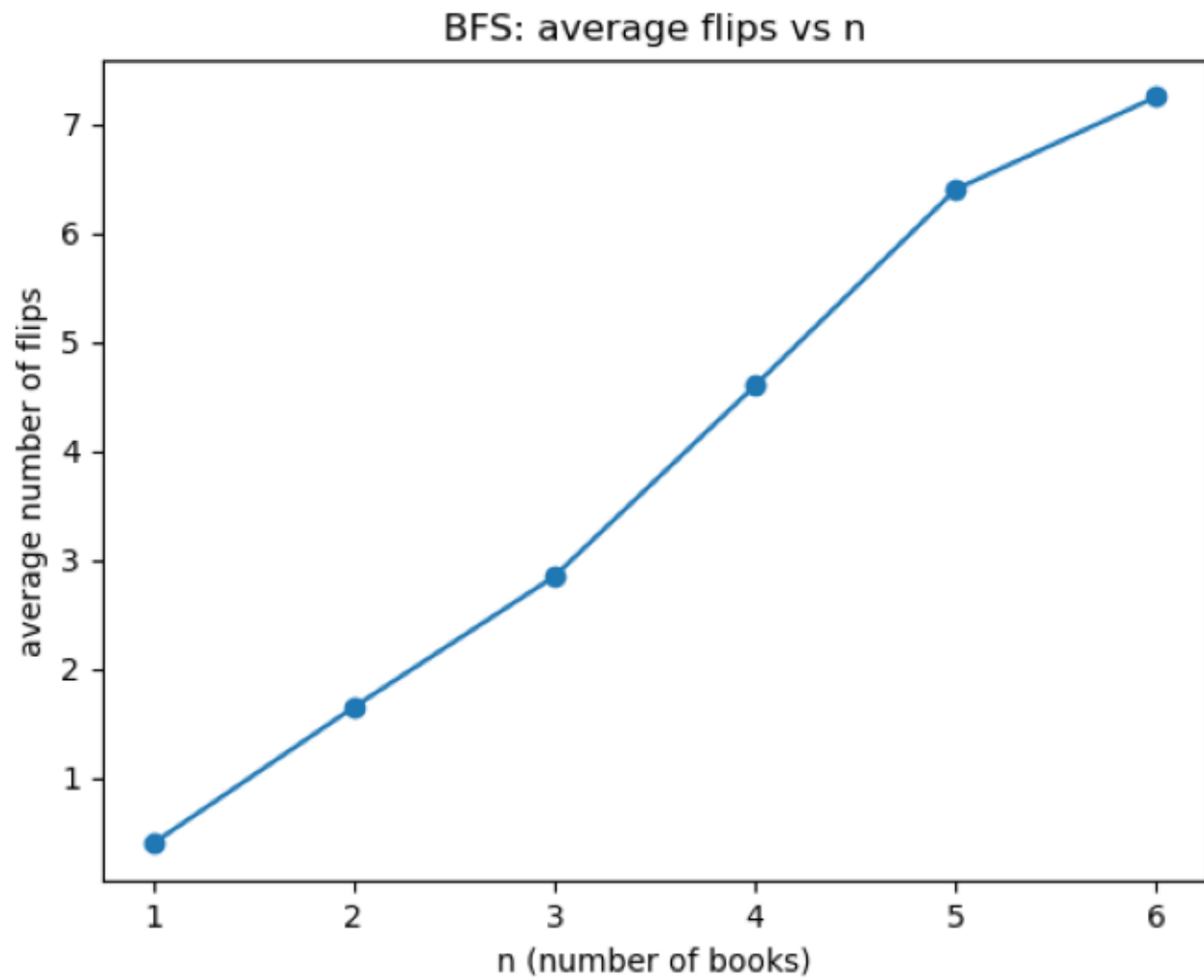
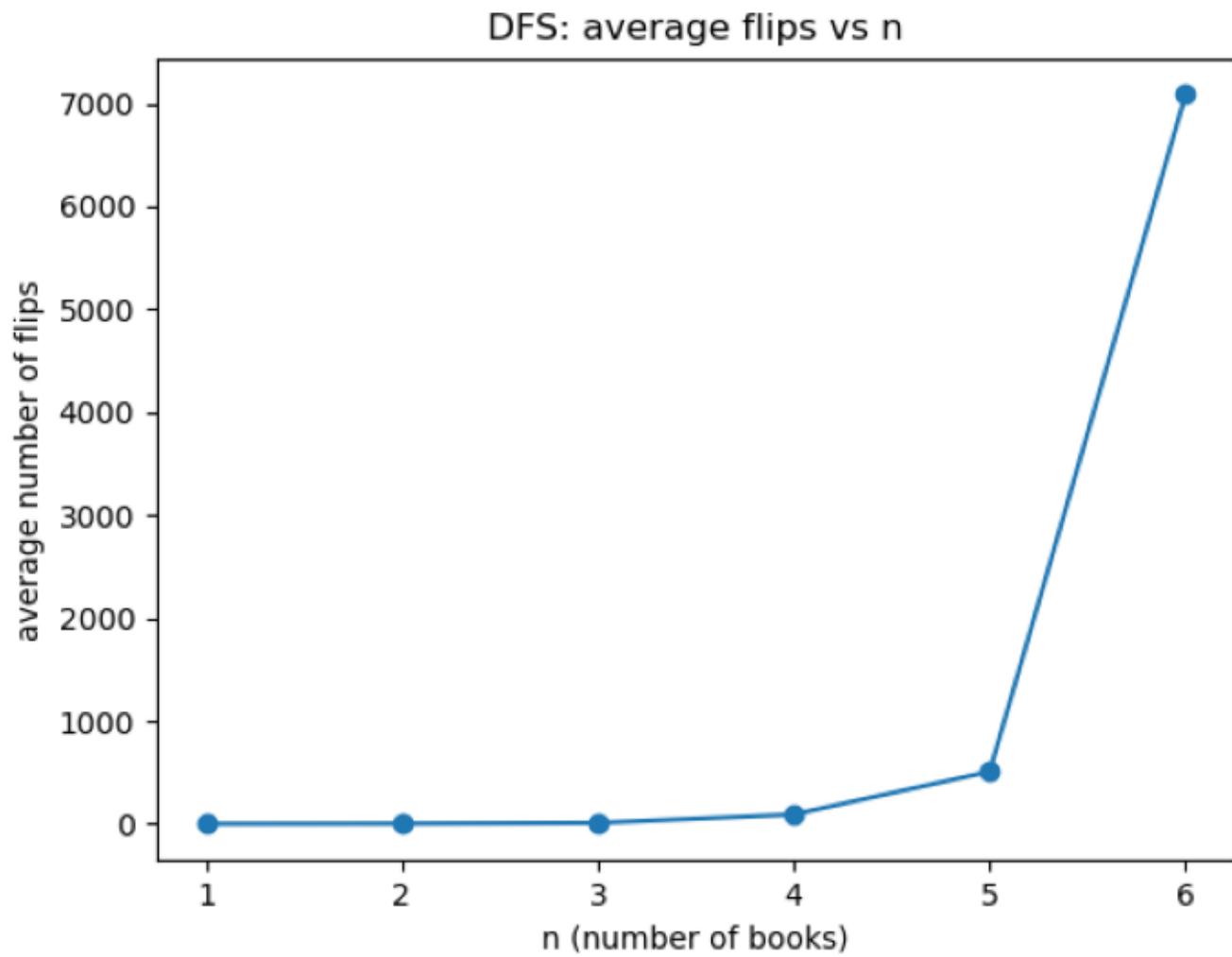


BFS: Average Number of Flips vs n



DFS: Average Number of Flips vs n



Function: average_flips_by_n (from lab1_extra_credit.py)

```
def average_flips_by_n(search_function, n_values, num_samples):
    # returns avg flips for each n
    average_flips_list = []

    for n in n_values:
        total_flips = 0

        for sample_num in range(num_samples):
            # make one random stack of size n
            random_order = list(range(n))
            random.shuffle(random_order)
            random_orientations = [random.randint(0, 1) for i in range(n)]

            random_stack = TextbookStack(random_order, random_orientations)

            # run the search, count its flips
            flip_sequence = search_function(random_stack)
            total_flips = total_flips + len(flip_sequence)

        # average over all samples for this n
        average_flips = total_flips / num_samples
        average_flips_list.append(average_flips)

    return average_flips_list
```

Lab 1 Extra Credit: Analysis and Discussion

For each n from 1 to 6, I made 20 random textbook stacks (random order and random orientations) and ran both breadth_first_search and depth_first_search on each one. Then I averaged the number of flips it took to sort the stack, for each n .

Results (average flips):

$n = 1$:	BFS = 0.40	DFS = 0.45
$n = 2$:	BFS = 1.65	DFS = 2.75
$n = 3$:	BFS = 2.85	DFS = 9.30
$n = 4$:	BFS = 4.60	DFS = 91.75
$n = 5$:	BFS = 6.40	DFS = 507.90
$n = 6$:	BFS = 7.25	DFS = 7086.50

Discussion:

BFS's average number of flips grows slowly and stays small (under 8 flips even at $n = 6$). This makes sense because BFS always finds the shortest possible flip sequence: it explores the search space level by level, so the first time it reaches a sorted stack, that path is guaranteed to be one of the shortest ones.

DFS's average number of flips grows extremely fast, blowing up to over 7000 flips by $n = 6$. This is because DFS dives all the way down one branch of the search tree before backtracking, with no guarantee that the branch it picks leads anywhere close to the goal. Since it does not compare path lengths, it can wander through a huge part of the state space before it happens to reach a sorted stack, especially as n grows and the state space ($n!$ orderings times 2^n orientations) grows very quickly.

Overall, this shows the classic tradeoff between BFS and DFS: BFS gives the shortest (optimal) solution but needs more memory to store the frontier, while DFS uses less memory per step but its solution length is unpredictable and can become very long as the problem size increases.